1 Pseudocode notation

In this text we shall write our algorithms using pseudocode. The idea behind pseudocode is to be "language independent" so that we can express what an algorithm does without needing to re-implement the ideas in every language. The pseudocode that we write will most similar to Python, but the ideas should be abstract enough to easily be translated into other languages. We note that we may use several different programming paradigms as needed to express our code concisely. What follows is a list of some of the operations and notations that will be seen in our pseudocode.

Assignment We can assign values to variables. There are two standard notations for this: either $x = 0$ or $x \leftarrow 0$. In either case the value on the right hand side is assigned to the variable on the left hand side.

Basic arithmetic We assume that all of the basic arithmetic operations such as addition, subtraction, division, multiplication, and exponentiation are possible $(+, -, /, \cdot, ^{)}$.

Functions We can define functions that take any number of inputs and can return values. These functions can be called by using their name and providing sufficient inputs. Functions can return any number of values or variables (including 0).

Conditional statements We can perform the basic if, else-if, and else statements that are present in most programming languages. We assume that an if-elseif-else must contain an if statement, can contain any number of else-if statements, and at most one else statement. The conditions can be any proposition (i.e. something that evaluates unambiguously to true or false). This includes that we can compare values using standard binary comparison operations $(<, \leq, =, \geq, >)$

```
if n ≤ 10 then
    Do something                                   ▷ Occurs when n ≤ 10
else if n ≤ 20 then
    Do something else.                             ▷ Occurs when n ≤ 10 and n ≰ 10
else if n ≥ 50 then
    Do something else.                             ▷ Occurs when n ≥ 50 and n ≰ 20 and n ≰ 10
else
    Do something else.                             ▷ Occurs when none of the above conditions are satisfied
end if
```

For loops We use a for loop to execute a particular chunk of code more than once. A standard for loop introduces a loop variable, the starting value of that variable, and the final value of the loop variable. The body of the for loop is the code that resides between the **for** and **end for** statements; the body executes once for every value the loop variable takes.

```
for i = 1 to 50 do
    x = x + i                                      ▷ This line executes once for each value i = 1, 2, 3, ..., 50
end for
```

For each loops We can use a for-each loop to iterate over every value in some collection. The only difference between the for loop and the for each loop is that the for each loop allows to iterate over any collection..

```
foreach x ∈ S do
    Do something with x                    ▷ Executes once for every x ∈ S
end for
```

```
i = 1
while i ≤ 2000 do
    x = x + i
    i = 3i
end while
```

While loops We can use a while loop to repeat a chunk of code until some Boolean condition is satisfied.

Repeat until loops A repeat until loop is nearly identical to a while loop, but it will always execute at least once.

```
i = 1
repeat
    x = x + i
    i = 3i
until i > 2000
```

To ensure that the reader is comfortable with pseudocode we will examine some simple code snippets and discuss their function.

Example 1. Consider the following code,

```
x ← 0
i ← 1
while i ≤ 100 do
    x ← x + i
    i ← 2i
end while
```

What does the code do? What is the value of i once the code finishes execution? How about the value of x?

Solution 1. The code creates two variables x and i with initial values of 0 and 1 respectively. Since $i = 1 \leq 100$, the code will enter the while loop. To trace the values of the variables, we shall create a simple table that stores those values over time. The code will update the value of x and then multiply i by 2. The values of x and i will be

Value of x	Value of i
0	1
1	2
3	4
7	8
15	16
31	32
63	64
127	128

Once $i = 128 > 100$, the while loop will cease; therefore, values of x and i when the code finishes execution are

$$x = 127 \qquad\qquad i = 128 \tag{1}$$

Example 2. Consider the code snippet below

```
function Adder(x,y)
    result ← x + y
    return result
end function
a ← 5
b ← 23
c ← Adder(a, b)
d ← Adder(c, Adder(a, a^2))
```

What does the function `Adder` do? What is the value of c? What about d?

Solution 2. The function `Adder` takes two values x and y as input and returns $x + y$. Therefore,

$$c = 5 + 23 = 28 \tag{2}$$

and $\texttt{Adder}(a, a^2) = 5 + 25 = 30$, so

$$d = 28 + 30 = 58 \tag{3}$$

Example 3. Consider the code snippet

```
function Func(n)
    x ← 0
    for i ← 1 to n do
        for j ← 1 to i do
            x ← x + i − j
        end for
    end for
    return x
end function
```

What does the function do? How many times does the statement $x \leftarrow x + i - j$ execute?

Solution 3. The code initializes a variable x to be 0, but directly computing what the for loops do to x seems difficult. Here, the loops are nested within each other and there is some interdependence between them. When $i = 1$ the inner loop (the loop which depends upon j) executes once (since j goes from 1 to 1). When $i = 2$, the inner loop executes twice (since j goes from 1 to 2). In general, the inner loop executes i times and we already know that i goes from 1 to n. So, we can compute the number of times that line $x \leftarrow x + i - j$ executes by computing the following summation,

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}. \tag{4}$$

As we will see later, summations and loops are very closely related and we will almost always use summations to determine how many times a loop executes!

Motivation

The purpose of this course (and courses like it) is to help the reader to develop more efficient algorithms. In order to develop more efficient algorithms is stands to reason that we should have some sensible definition of efficient and a sensible means of comparing algorithms. For example, consider the following (pointless) algorithm

```
for i = n to 3n^2 + 4n do
    for j = 1 to 7n + 3 do
        x = x + i - j
    end for
end for
```

Algorithm 1: A pointless code snippet

Can we determine the running time of this algorithm? In practice we can always determine the running time of an algorithm by simply timing it. If we get the time just before the algorithm begins and the time after the algorithm terminates and then compute their difference we can acquire the running time. Unfortunately the pseudocode that we have written here is not timable since there is an unknown quantity, n, within the algorithm. Let us try and rephrase the question more concretely: can we determine the running time of the algorithm as a function of n? This more general question allows to understand the algorithm at a higher level of abstraction.

In order to understand the running time of *any piece of code* we must first understand how long the various operations take. How long does it take to access a variable that is stored in memory? How long does it take to add a pair of numbers? What about subtraction? Multiplication? Addition? Assignment of a variable? Accessing the elements of an array? For each of these "fundamental" operations we must have an understanding of how long those operations take to occur. Do the operations depend upon the values provided as input? Does it take more time to add large values than it does to add small values? Is it possible addition of certain values is faster than other values? Deciding upon the answers to these questions (among many others) is a critical foundation to our analysis; declaring these assumptions lets us develop a "model of computation" that we can then use to analyze any algorithm. Different models of computation may yield different running times or even allow for the implementation of different algorithms. For the purposes of this text we shall make the following fundamental assumptions about the code that we write

1. The basic binary arithmetic operations take constant time $(+, -, *, /, (\cdot)^n)$.
2. The basic binary comparison operations take constant time $(<, \leq, =, \geq, >)$
3. Variables can be accessed and assigned in constant time.
4. Values can be returned by a function in constant time.
5. Given an array A containing values $A[1], A[2], \ldots, A[n]$, we can access and assign values to an arbitrary element in that array $A[i]$ in constant time (this is sometimes referred to as Random Access Lookup).
6. We have access to an infinite amount of memory.
7. We can create functions/procedures/subroutines

One can argue whether these assumptions are reasonable and a more formal discussion of these ideas can be found

in many texts (TODO: references), but for now we will take for granted that these assumptions are reasonable.

In general, we can compute the running time of an algorithm[1] by finding the sum of the costs of each line of code. Some lines of code in algorithm may execute multiple times this sum may include the same line of code many times over, but the principle remains the same. For example, in Algorithm 1 line 3 executes many times. Given our model of computation, we observe that line 3 of Algorithm 1 takes a constant amount of time. To demonstrate this fact consider the following method of computing $x = x + i - j$

1. The algorithm accesses x and i, then computes and stores $x + i$ which takes $2t_{acc} + t_{store} + t_{add}$.
2. The algorithm then accesses $x + i$ and j. then computes $x + i - j$ and stores that value in place of x which takes $2t_{acc} + t_{store} + t_{sub}$.

Since the time to access, store, add, and subtract values are all constants the total running time of line 3 $4t_{acc} + 2t_{store} + t_{add} + t_{sub}$ and each of these operations is assumed to take constant time. For now, let us take a higher level look at the code and ignore the specifics of the computation. As we mentioned, the line takes constant time, so let c_3 be the time required to execute line 3 once. In order to determine the time required by Algorithm 1 still need to determine how many times line 3 executes.

In order to compute the number of times that line 3 executes we begin by computing the number of occurrences of the for loop on lines 2-4. The body of the for loop executes once for every value of $j = 1, 2, 3, \ldots, 7n + 3$. Depending on how the psuedocode is implemented line 2 of Algorithm 1 may potentially execute once for the value $j = 7n + 4$ and then determine that $7n + 4 > 7n + 3$ and not enter the body of the loop. The purpose of this text is a high level approach to analyze algorithms and to not get bogged down in the specifics (not that these specifics are unimportant, but they are not our focus), so we shall simply assume that the for loop header executes exactly as many times as the body of the for loop.

So, now we want to "add up" the cost of running lines 2-4 for each value that j takes. (TODO: Make an entire section on summation analysis) That is,

$$\text{Cost of executing lines 2-4 once} = \sum_{j=1}^{7n+3} c = c(7n + 3) \tag{5}$$

Note that this running time ignores the outer for loop. Further, we note that this cost is the cost of a single iteration of the outer for loop; therefore, we must apply the exact same process to determine how long the outer for loop takes. So,

$$\text{Cost of executing lines 1-5} = \sum_{i=n}^{3n^2+4n} c(7n + 3) = c(3n^2 + 3n + 1)(7n + 3) \tag{6}$$

Therefore, the running time of Algorithm 1 is $c(3n^2 + 3n + 1)(7n + 3)$. We note that this form is very awkward to deal with and that directly comparing this to other algorithms may prove difficult since we have captured the running time to an almost extreme degree of exactness. To deal with this issue we shall introduce a more

[1] When we say algorithm, we mean any sequence of operations that can be performed using the above assumptions.

sophisticated notation to more easily compare and contrast different algorithms for solving the same problem. For example, we notice that in the expression

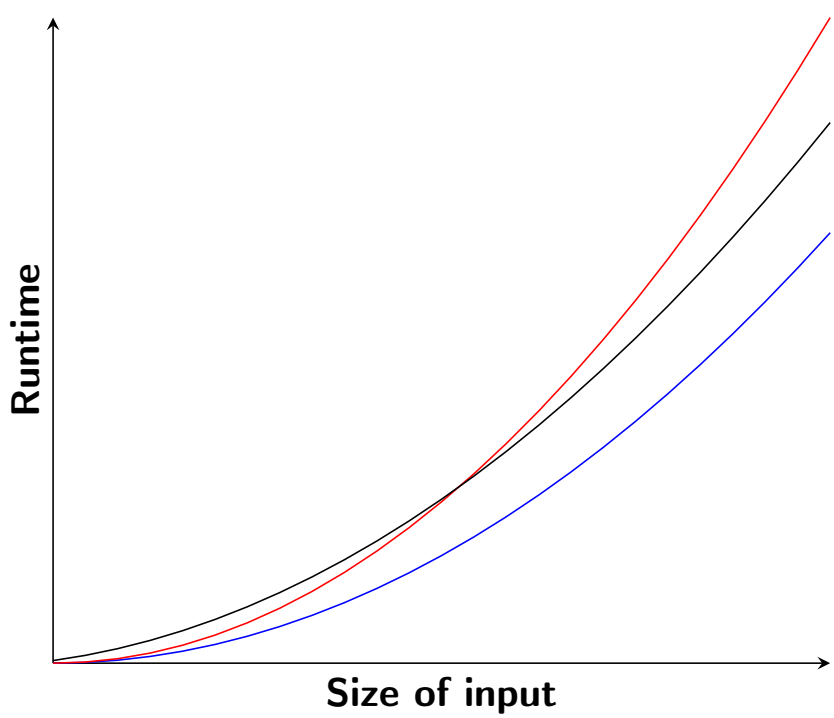
$$c(3n^2 + 3n + 1)(7n + 3) = c(21n^3 + 30n^2 + 16n + 3)$$

the term that will affect the growth rate the most is the $21cn^3$ term and will say

$$c(21n^3 + 30n^2 + 16n + 3) \in \Theta(n^3)$$

to mean that "the expression grows like n^3". This notation will prove very convenient going forward – we will formally define what this notation means and then prove various properties about it.

Intuitively, we wish that when we say $f(n) \in \Theta(g(n))$ that we can say "$f(n)$ looks like $g(n)$", but with some formal notion of "looks like". Developing useful and powerful definitions is more of an art than a science, but it is worthwhile to try this on your own. Come up with some definition and see where that definition leads you. Try some other options and see what results follow.



Introduction and Definitions

Definition 1. A function $f(n)$ is in the set $O(g(n))$ if there are constants $c > 0$ and $n_0 \in \mathbb{N}$ such that for all $n > n_0$, $f(n) \leq cg(n)$.

Definition 2. A function $f(n)$ is in the set $\Omega(g(n))$ if there are constants $c > 0$ and $n_0 \in \mathbb{N}$ such that for all $n > n_0$, $f(n) \geq cg(n)$.

Definition 3. A function $f(n)$ is in the set $\Theta(g(n))$ if there are constants $c_1 > 0$ and $c_2 > 0$ and $n_0 \in \mathbb{N}$ such that for all $n > n_0$, $c_1 g(n) \leq f(n) \leq c_2 g(n)$.

Definition 4. A function $f(n)$ is in the set $o(g(n))$ if $f(n) \in O(g(n))$ and $f(n) \notin \Omega(g(n))$.

Definition 5. A function $f(n)$ is in the set $\omega(g(n))$ if $f(n) \in \Omega(g(n))$ and $f(n) \notin O(g(n))$.

1. If $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$, then $f(n) \in O(g(n))$ and $f(n) \notin \Omega(g(n))$
2. If $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$, then $f(n) \in \Omega(g(n))$ and $f(n) \notin O(g(n))$
3. If $\lim_{n\to\infty} \frac{f(n)}{g(n)} = c > 0$, then $f(n) \in \Theta(g(n))$

Hierarchy of Asymptotic Complexity

$\Theta(n^n)$

$\Theta(n!)$

$\Theta(3^n)$

$\Theta(2^n)$

$\Theta(n^3)$

$\Theta(n^2)$

$\Theta(n\log(n))$

$\Theta(n)$

$\Theta(\sqrt{n})$

$\Theta(n^{0.1})$

$\Theta((\log(n))^2)$

$\Theta(\log(n))$

$\Theta(1)$

Examples

Example 4. Show $\sqrt{6n^3 + 7n^2 + 3n + 5} \in \Theta(n^{1.5})$

Proof. We can prove that $\sqrt{6n^3 + 7n^2 + 3n + 5} \in \Theta(n^{1.5})$ by showing $\sqrt{6n^3 + 7n^2 + 3n + 5} \in O(n^{1.5})$ and $\sqrt{6n^3 + 7n^2 + 3n + 5} \in \Omega(n^{1.5})$.

$\sqrt{6n^3 + 7n^2 + 3n + 5} \in O(n^{1.5})$ We observe the following facts

$$\begin{aligned} 7n^2 &\leq 7n^3 && \text{when } n \geq 0 \\ 3n &\leq 3n^3 && \text{when } n \geq 0 \\ 5 &\leq 5n^3 && \text{when } n \geq 1 \end{aligned}$$

therefore

$$\sqrt{6n^3 + 7n^2 + 3n + 5} \leq \sqrt{6n^3 + 7n^3 + 3n^3 + 5n^3} \leq \sqrt{21n^3} \leq \sqrt{21}n^{1.5}$$

when $n \geq 1$.

$\sqrt{6n^3 + 7n^2 + 3n + 5} \in \Omega(n^{1.5})$ We observe the following facts

$$\begin{aligned} 7n^2 &\geq 0 && \text{when } n \geq 0 \\ 3n &\geq 0 && \text{when } n \geq 0 \\ 5 &\geq 0 && \text{when } n \geq 0 \end{aligned}$$

therefore

$$\sqrt{6n^3 + 7n^2 + 3n + 5} \geq \sqrt{6n^3} \geq \sqrt{6}n^{1.5}$$

when $n \geq 0$.
Thus, when $n \geq 1$

$$\sqrt{6}n^{1.5} \leq \sqrt{6n^3 + 7n^2 + 3n + 5} \leq \sqrt{21}n^{1.5}$$

and we can conclude that $\sqrt{6n^3 + 7n^2 + 3n + 5} \in \Theta(n^{1.5})$.

□

Example 5. Show $\sqrt{6n^3 - 7n^2 + 3n + 5} \in \Theta(n^{1.5})$

Proof. We can prove that $\sqrt{6n^3 - 7n^2 + 3n + 5} \in \Theta(n^{1.5})$ by showing $\sqrt{6n^3 - 7n^2 + 3n + 5} \in O(n^{1.5})$ and $\sqrt{6n^3 - 7n^2 + 3n + 5} \in \Omega(n^{1.5})$.

$\sqrt{6n^3 - 7n^2 + 3n + 5} \in O(n^{1.5})$ We observe the following facts

$$\begin{aligned} -7n^2 &\leq 0 && \text{when } n \geq 0 \\ 3n &\leq 3n^3 && \text{when } n \geq 0 \\ 5 &\leq 5n^3 && \text{when } n \geq 1 \end{aligned}$$

therefore

$$\sqrt{6n^3 - 7n^2 + 3n + 5} \leq \sqrt{6n^3 + 0 + 3n^3 + 5n^3} \leq \sqrt{14n^3} \leq \sqrt{14}n^{1.5}$$

when $n \geq 1$.

$\sqrt{6n^3 - 7n^2 + 3n + 5} \in \Omega(n^{1.5})$ We observe the following facts

$$\begin{aligned} -7n^2 &\geq -n^3 && \text{when } n \geq 7 \\ 3n &\geq 0 && \text{when } n \geq 0 \\ 5 &\geq 0 && \text{when } n \geq 0 \end{aligned}$$

therefore

$$\sqrt{6n^3 - 7n^2 + 3n + 5} \geq \sqrt{6n^3 - n^3} \geq \sqrt{5}n^{1.5}$$

when $n \geq 7$.
Thus, when $n \geq 7$

$$\sqrt{5}n^{1.5} \leq \sqrt{6n^3 + 7n^2 + 3n + 5} \leq \sqrt{14}n^{1.5}$$

and we can conclude that $\sqrt{6n^3 - 7n^2 + 3n + 5} \in \Theta(n^{1.5})$.

□

Example 6. Use the limit theorem to determine how the following pairs of functions compare asymptotically:

1. $2n^4 + 4n^3 + n^2$ vs $9n^3 + 7n^2 + 6n$

$$\lim_{n\to\infty} \frac{2n^4 + 4n^3 + n^2}{9n^3 + 7n^2 + 6n} = \lim_{n\to\infty} \frac{2n + 4 + \frac{1}{n}}{9 + \frac{7}{n} + \frac{6}{n^2}} = \lim_{n\to\infty} \frac{2n+4}{9} = \infty$$

   Therefore, $(2n^4 + 4n^3 + n^2) \in \Omega(9n^3 + 7n^2 + 6n)$.

2. $n^{1/2}$ vs $n^{1/4}$

$$\lim_{n\to\infty} \frac{n^{1/2}}{n^{1/4}} = \lim_{n\to\infty} n^{1/4} = \infty$$

   Therefore, $n^{1/2} \in \Omega(n^{1/4})$.

3. $\log_2(n)$ vs $\log_3(n)$

$$\lim_{n\to\infty} \frac{\log_2(n)}{\log_3(n)} = \lim_{n\to\infty} \frac{\frac{\log_3(n)}{\log_3(2)}}{\log_3(n)} = \lim_{n\to\infty} \frac{1}{\log_3(2)} = \frac{1}{\log_3(2)}$$

   Therefore, $\log_2(n) \in \Theta(\log_3(n))$.

4. $\log_2(n)$ vs $\log_2(n^2)$

$$\lim_{n\to\infty} \frac{\log_2(n)}{\log_2(n^2)} = \lim_{n\to\infty} \frac{\log_2(n)}{2\log_2(n)} = \lim_{n\to\infty} \frac{1}{2} = \frac{1}{2}$$

   Therefore, $\log_2(n) \in \Theta(\log_2(n^2))$.

5. $\log_2(n)$ vs $(\log_2(n))^2$

$$\lim_{n\to\infty} \frac{\log_2(n)}{(\log_2(n))^2} = \lim_{n\to\infty} \frac{1}{\log_2(n)} = 0$$

   Therefore, $\log_2(n) \in O((\log_2(n))^2)$.

6. $\log_2(n)$ vs $n^{0.1}$

$$\begin{aligned}
&\lim_{n\to\infty} \frac{\log_2(n)}{n^{0.1}} = \frac{\infty}{\infty} \\
&\overset{LHR}{=} \lim_{n\to\infty} \frac{\frac{1}{\ln(2)n}}{0.1n^{-0.9}} = \lim_{n\to\infty} \frac{1}{0.1\ln(2)n^{0.1}} = 0
\end{aligned}$$

   Therefore, $\log_2(n) \in O(n^{0.1})$.

7. n^3 vs 3^n

$$\begin{aligned}
&\lim_{n\to\infty} \frac{n^3}{3^n} = \frac{\infty}{\infty} \\
&\overset{LHR}{=} \lim_{n\to\infty} \frac{3n^2}{\ln(3)3^n} = \frac{\infty}{\infty} \\
&\overset{LHR}{=} \lim_{n\to\infty} \frac{6n}{\ln(3)^2 3^n} = \frac{\infty}{\infty} \\
&\overset{LHR}{=} \lim_{n\to\infty} \frac{6}{\ln(3)^3 3^n} = \frac{6}{\ln(3)^3} \lim_{n\to\infty} \left(\frac{1}{3}\right)^n = 0
\end{aligned}$$

   Therefore, $n^3 \in O(3^n)$.

8. n vs $n\log_2(n)$

$$\lim_{n\to\infty} \frac{n}{n\log_2(n)} = \lim_{n\to\infty} \frac{1}{\log_2(n)} = 0$$

Therefore, $n \in O(n\log_2(n))$

9. n^2 vs $n\log_2(n)$

$$\lim_{n\to\infty} \frac{n^2}{n\log_2(n)} = \lim_{n\to\infty} \frac{n}{\log_2(n)} = \infty$$

Therefore, $n^2 \in \Omega(n\log_2(n))$.

10. n vs $\log_2(3^n)$

$$\lim_{n\to\infty} \frac{n}{\log_2(3^n)} = \lim_{n\to\infty} \frac{n}{n\log_2(3)} = \lim_{n\to\infty} \frac{1}{\log_2(3)} = \frac{1}{\log_2(3)}$$

Therefore, $n \in \Theta(\log_2(3^n))$.

Example 7. Give the asymptotic complexity of each of the following functions in simplest terms. Your solution should have the form $\Theta(n^\alpha)$ or $\Theta(\log_\gamma(n)^\beta)$ or $\Theta(n^\alpha \log_\gamma(n)^\beta)$ or $\Theta(\alpha^n)$ or $\Theta(1)$.

1. $3n^2 + 8n - 2 \in \Theta(n^2)$
2. $(2n^3 + 6)^4 \in \Theta(n^{12})$
3. $\log_3(n^2 + 2^n) + \sqrt{n} \in \Theta(n)$
4. $5^{\log_5(n)} = n \in \Theta(n)$
5. $25^{\log_5(n)} = n^2 \in \Theta(n^2)$

```
function Func1(n)
    x = 0
    for i = 1 to n do
        for j = 1 to i do
            x = x + i − j
        end for
    end for
    return x
end function
```

We can express the running time of this algorithm as a summation. Let $T(n)$ be the running time of `Func1` with input n, then

$$T(n) = \sum_{i=1}^{n} \sum_{j=1}^{i} c = \sum_{i=1}^{n} ci = c \sum_{i=1}^{n} i = c\frac{n(n+1)}{2} \in \Theta(n^2)$$

Therefore, the algorithm takes $\Theta(n^2)$ time.

```
1:  function Func2(n)
2:      if n > 100000 then
3:          return 0
4:      end if
5:      x = 0
6:      for i = 1 to n do
7:          for j = 1 to i do
8:              x = x + i − j
9:          end for
10:     end for
11:     return x
12: end function
```

For sufficiently large values of n (that is when $n > 100000$) the algorithm will simply enter the If statement on line 2 and immediately return. Therefore, the algorithm takes $\Theta(1)$ time.

```
function Func3(n)
    if n < 100000 then
        return 0
    end if
    x = 0
    for i = 1 to n do
        for j = 1 to i do
            x = x + i − j
        end for
    end for
    return x
end function
```

For sufficiently large values of n the algorithm *never* enters the If statement and therefore is identical to `Func1`. So, the algorithm takes $\Theta(n^2)$ time.